



Pashov Audit Group

PAXGy Chainlink CCIP Integration Security Review

August 26th 2026 - August 28th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. Methodology	4
5. Executive Summary	4
6. Findings	4
Info Severity	5
[I-01] CCIP selector remap leaves the destination teller stale, burning outbound shares into an unrecoverable no-op delivery	5
[I-02] Lane lifecycle does not clear CCIP per-lane state, so re-provisioned lanes inherit stale configuration	7
[I-03] CCIP inbound capacity can permanently reject a bridge message larger than the destination bucket	8



1. About Pashov Audit Group

Pashov Audit Group consists of 100+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 500 security audits completed - uncovering and helping patch thousands of vulnerabilities - the group strives to create the most robust and pleasant audit experience possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Telegram [@pashovkrum](#).

2. Disclaimer

A security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where the researchers try to find as many issues as possible. We can not guarantee 100% security after the review. Subsequent reviews, bug bounty programs and on-chain monitoring are strongly recommended. The client's remediation of each issue has been reviewed; the outcome is recorded in the finding's Resolution section.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Critical

Loss or compromise of critical assets, or protocol-wide impact.

High

Significant material loss of assets, or serious harm to a group of users.

Medium

Moderate material loss, or moderate harm, often under conditional assumptions.

Low

Minor impact, limited scope, or requires unlikely conditions to trigger.

Info

No direct security impact - best-practice, code-quality or hardening suggestion.

IMPACT

High

- leads to a significant material loss of assets in the protocol or significantly harms a group of users

Medium

- leads to a moderate material loss of assets in the protocol or moderately harms a group of users

Low

- leads to a minor material loss of assets in the protocol or harms a small group of users

LIKELIHOOD

High

- attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost

Medium

- only a conditionally incentivized attack vector, but still relatively likely



Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. Methodology

A large team of security researchers hunt vulnerabilities on the scope provided during a time-boxed review.

Every issue submission is deduplicated and validated by a judging team. Every accepted issue appears below with its impact and a recommended remediation. The client's response to each issue was then reviewed, and its resolution status - fixed, acknowledged, or invalidated - is recorded with the finding.

5. Executive Summary

A time-boxed security review of PAXGy Chainlink CCIP Integration was conducted by the Pashov Audit Group team. This report documents 3 issues across the reviewed codebase, reported by BengalCatBalu, BRDNS, farismaulana, Hala, IvanFitro, shaka, trtrth, unforgiven.

SCOPE

<https://github.com/paxoslabs/pxl-boring-vault/pull/347>

REVIEW COMMIT HASH

[76db7f6c7de717bdf6ccbaa922c4ab5102a7c963](#) (paxoslabs/pxl-boring-vault)

FIXES REVIEW COMMIT HASH

[164d790322f51784cc2406408d5e9f290e7780f7](#) (paxoslabs/pxl-boring-vault)

6. Findings

ID	TITLE	SEVERITY	STATUS
I-01	CCIP selector remap leaves the destination teller stale, burning outbound shares into an unrecoverable no-op delivery	Info	Fixed
I-02	Lane lifecycle does not clear CCIP per-lane state, so re-provisioned lanes inherit stale configuration	Info	Acknowledged
I-03	CCIP inbound capacity can permanently reject a bridge message larger than the destination bucket	Info	Acknowledged



Info Severity

[I-01] CCIP selector remap leaves the destination teller stale, burning outbound shares into an unrecoverable no-op delivery

Info

Fixed

SUMMARY

A bridge lane's identity lives in two independently updated storage locations: the physical destination selector (`chainSelectorToCcipSelector[key]`, set by `setCcipChainSelector`) and the destination receiver (`selectorToChains[key].targetTeller`, set by the base-teller chain setters).

`setCcipChainSelector` only touches the selector mappings and never updates the Chain row:

```
chainSelectorToCcipSelector[nucleusChainSelector] = ccipChainSelector;  
ccipSelectorToChainSelector[ccipChainSelector] = nucleusChainSelector;
```

So an admin can repoint a lane's destination to a new chain while `targetTeller` still holds the previous chain's teller, leaving the two halves of the lane inconsistent.

`bridge()` validates only the Chain row, then burns the user's shares before dispatch. Because the stale row's `targetTeller` and allow flags are internally consistent, all checks pass:

```
vault.exit(address(0), ERC20(address(0)), 0, msg.sender, shareAmount); // burn happens here  
messageId = _bridge(shareAmount, data);
```

The outbound message then mixes the two locations: the destination comes from the freshly remapped selector, but the receiver comes from the unchanged Chain row.

```
receiver: abi.encode(selectorToChains[data.chainSelector].targetTeller), // stale teller
```

That teller address has no meaning on the chain the selector now points to. In the common case it holds no code (`receiver.code.length == 0`); even if some unrelated contract occupies the address, it will not advertise `IAny2EVMMMessageReceiver`. Either branch makes the CCIP OffRamp classify the data message as a token-only transfer, skip `ccipReceive`, drop the payload, and mark the message SUCCESS.

```
return (dataLength == 0 && ccipReceiveGasLimit == 0) || receiver.code.length == 0  
    || !receiver._supportsInterfaceReverting(type(IAny2EVMMMessageReceiver).interfaceId);
```

Because the delivery succeeds it never enters the FAILED state, so there is nothing to re-execute and the burned shares are permanently lost. The same outcome occurs whenever a key is remapped to any chain where the protocol has not deployed the teller, or where the teller lives at a different address than the stale row records.

IMPACT

An authorized `setCcipChainSelector` change that precedes or races a user bridge causes the user's burned shares to be dispatched to an address that is not a valid teller on the destination chain, where the message is dropped as a no-op.



The loss is permanent and equals the full bridged share amount, not merely the transport fee. There is no FAILED state to retry, no source-side refund, and no claim path.

Any bridge transaction already sitting in the mempool when the remap lands is also exposed, so affected users are not limited to those acting after the change.

EXPLOIT PATH

1. Lane key 2 maps to Base, with `targetTeller = baseTeller`.
2. An admin calls `setCcipChainSelector(2, arbitrumSelector)`, repointing key 2 to Arbitrum while the Chain row for key 2 still holds `baseTeller`.
3. A user bridges to key 2, still believing it means Base. `_beforeBridge` passes against the stale row and `vault.exit` burns the shares.
4. The message is sent to Arbitrum with `receiver = baseTeller`, which holds no code there (or hosts an unrelated contract that does not implement `IAny2EVMMMessageReceiver`).
5. The OffRamp drops the payload as a token-only transfer and marks the message SUCCESS. No shares are minted anywhere and the burn is unrecoverable.

PREREQUISITES

Requires the authorized holder of `setCcipChainSelector` to remap an active lane, whether by operational mistake or expected reconfiguration, while at least one bridge to that key is in flight or subsequently submitted.

RECOMMENDATION

Make it impossible to hold the two halves of a lane live and inconsistent. Reject `setCcipChainSelector` while the lane is still enabled, forcing an explicit disable-drain-remap procedure:

```
function setCcipChainSelector(uint32 nucleusChainSelector, uint64 ccipChainSelector) external
requiresAuth {
    if (nucleusChainSelector == 0 || ccipChainSelector == 0) revert InvalidChainSelector();

    + Chain memory chain = selectorToChains[nucleusChainSelector];
    + if (chain.allowMessagesFrom || chain.allowMessagesTo) revert
CcipChainStillActive(nucleusChainSelector);

    uint64 oldCcipSelector = chainSelectorToCcipSelector[nucleusChainSelector];
    if (oldCcipSelector != 0) delete ccipSelectorToChainSelector[oldCcipSelector];

    uint32 oldNucleusSelector = ccipSelectorToChainSelector[ccipChainSelector];
    if (oldNucleusSelector != 0) delete chainSelectorToCcipSelector[oldNucleusSelector];

    chainSelectorToCcipSelector[nucleusChainSelector] = ccipChainSelector;
    ccipSelectorToChainSelector[ccipChainSelector] = nucleusChainSelector;

    emit CcipChainSelectorSet(nucleusChainSelector, ccipChainSelector);
}
```

RESOLUTION

Paxos Labs

fixed-PAG

Remediation PR: <https://github.com/paxoslabs/pxl-boring-vault/pull/353>

[I-02] Lane lifecycle does not clear CCIP per-lane state, so re-provisioned lanes inherit stale configuration

Info

Acknowledged

SUMMARY

The adapter stores its per-lane CCIP state in six mappings

([MultiChainCCIPTellerWithMultiAssetSupport.sol:36-51](#)): the forward and reverse selector bijection, the outbound and inbound finality configs, and the outbound and inbound rate-limit buckets. The lane lifecycle in the base contract touches none of them.

`removeChain` deletes only the `Chain` struct:

```
function removeChain(uint32 chainSelector) external requiresAuth {
    delete selectorToChains[chainSelector];
    emit ChainRemoved(chainSelector);
}
```

`addChain` likewise writes only a fresh `Chain` struct, so overwriting an existing key leaves the six mappings intact. A complete cleanup exists in `resetCcipChainConfig` ([MultiChainCCIPTellerWithMultiAssetSupport.sol:171-187](#)), gated on both directions being disabled, but nothing in the lifecycle invokes it. The required discipline lives only in its docstring and is never enforced on-chain.

So a lane key that is removed and re-added for a different network keeps its old CCIP configuration. The new `Chain` struct names the new teller while `chainSelectorToCcipSelector[key]` still resolves to the old selector. The only routing guard rejects zero, not a mismatch:

```
function _ccipSelector(uint32 nucleusChainSelector) internal view returns (uint64 ccipSelector)
{
    ccipSelector = chainSelectorToCcipSelector[nucleusChainSelector];
    if (ccipSelector == 0) revert InvalidChainSelector();
}
```

A stale nonzero selector passes this check. The burn commits ([CrossChainTellerBase.sol:83](#)) and the message is dispatched toward the old network with the receiver encoding the new teller, which has no code on that network, so the destination mint never runs. The inherited finality configs and rate-limit buckets also bind the new lane, so an outbound bucket already exhausted in the lane's previous life reverts fresh bridges with `TokenRateLimitReached`.

IMPACT

While the desync stands, every bridge on the lane burns the caller's shares and spends the full quoted fee for a message addressed to a decommissioned network, where no mint ever occurs.

Nothing signals the mismatch: the struct-level checks in `_beforeBridge` pass, `previewFee` quotes a normal fee, and no revert or event flags the incomplete provisioning. Only a later explicit `setCcipChainSelector` repairs future bridges, and it does nothing for shares already burned.

Inherited rate-limit buckets can also cause the opposite failure mode, reverting otherwise valid bridges on the newly provisioned lane.



EXPLOIT PATH

1. An admin provisions a lane for network X: `addChain(...)` then `setCcipChainSelector(key, X)`, plus optional finality and rate-limit setters.
2. The admin retires the lane with `removeChain(key)`; the six CCIP mappings survive.
3. The admin re-adds the same key for network Y with a new teller via `addChain(key, ..., newTeller, ...)`.
4. A user bridges on the lane; shares burn and the fee is spent, but the message routes toward network X addressed to the new teller, which has no code there, so the mint never executes.

PREREQUISITES

Requires an admin to remove and later re-add the same lane key for a different network without calling `resetCcipChainConfig` in between. This is a plausible operational sequence because the cleanup is documented but not enforced.

RECOMMENDATION

Wire the adapter-side cleanup into the lane lifecycle through a virtual hook so a removed lane cannot leak state into a later re-add:

```
// src/base/Roles/CrossChain/MultiChainTellerBase.sol
function removeChain(uint32 chainSelector) external requiresAuth {
    delete selectorToChains[chainSelector];
+   _onChainRemoved(chainSelector);
    emit ChainRemoved(chainSelector);
}
+
+   function _onChainRemoved(uint32 chainSelector) internal virtual {}
```

Override the hook in the adapter with the same six deletes that `resetCcipChainConfig` already performs (`MultiChainCCIPtellerWithMultiAssetSupport.sol:177-184`). As defense in depth, have `addChain` reject a key whose `chainSelectorToCcipSelector` entry is still populated, forcing a re-pointed key through `setCcipChainSelector` explicitly.

RESOLUTION

Paxos Labs

acknowledged-PAG

This relies on the operator fundamentally misusing the chain ID and we acknowledge the desire to prevent misuse but the suggested fix requires changing otherwise unmodified parent contracts. This issue assumes we would label a chain ID as referring to 1 chain, then later updating that same chain ID to refer to a different chain. Which would never happen when using chain ID as a selector (as we do)

[I-03] CCIP inbound capacity can permanently reject a bridge message larger than the destination bucket

Info

Acknowledged

LOCATION

MultiChainCCIPtellerWithMultiAssetSupport:_bridge+ccipReceive:259-298



SUMMARY

The inbound and outbound CCIP rate limiters are independent buckets with potentially different capacities, and nothing guarantees that a message accepted on the source has a viable mint path on the destination.

The source `_bridge` consumes only the outbound bucket, then submits the message. On the destination, `ccipReceive` applies a separate inbound bucket whose capacity is a hard per-call upper bound. If the payload amount exceeds that capacity, `RateLimiter._consume` reverts with `TokenMaxCapacityExceeded`, even when the inbound bucket is completely full.

Waiting or retrying cannot help, because the bucket never refills above its configured capacity. For example, with the destination inbound capacity set to 10,000 shares and the source permitting 50,000, a 50,000-share message always reverts on `_consume(50,000)`.

IMPACT

A user can have their shares burned on the source chain while the corresponding CCIP message stays permanently unexecutable because its amount exceeds the destination inbound capacity.

No destination shares are minted, and recovery is possible only through administrative reconfiguration of the inbound capacity or another operational intervention.

EXPLOIT PATH

1. The destination inbound limiter is set to capacity 10,000 shares while the source outbound limiter is disabled or set to at least 50,000 shares.
2. A user calls bridge for 50,000 shares; `_beforeBridge` passes, `vault.exit` burns the shares, and the source router accepts the message.
3. The destination router calls `ccipReceive`, which reaches `_consume(50,000)` and reverts because capacity is 10,000.
4. Manual execution after waiting fails identically, leaving the burned shares with no destination mint until the inbound capacity is changed administratively.

PREREQUISITES

The destination inbound limiter must be enabled with a capacity lower than a source bridge amount that the source outbound path still permits.

RECOMMENDATION

Enforce a configured maximum compatible with the destination before burning:

```
uint256 destinationCapacity = configuredInboundCapacity[data.chainSelector];
if (destinationCapacity != 0 && shareAmount > destinationCapacity) {
    revert DestinationCapacityExceeded();
}
```

Preferably, derive or synchronize the source-side maximum from the destination lane configuration and reject any configuration update that would permit source messages larger than the destination inbound capacity.



RESOLUTION

Paxos Labs

acknowledged-PAG

We agree with Chainlink's interpretation that this requires misconfiguration and can always be retried along with a change in configuration. We intend to set the outbound and inbound configs equally so that it should be impossible for an inbound order size to exceed the capacity entirely while still being a valid order